STM32L452RE I²C Driver Development Course

Complete Lecture Notes

Polling · Interrupt · DMA

Lectures 9 – 23

Table of Contents

LECTURE 9: Introduction to Interrupt-Driven I²C Communication (STM32L452RE)

Objective

In the previous lectures, the I²C driver used polling (blocking). The CPU continuously waited for status flags such as TXIS, RXNE, and TC before proceeding.

Although simple, polling wastes CPU time because the processor remains occupied until the transfer finishes.

In this lecture, we introduce interrupt-driven I²C communication, where the CPU starts a transfer and continues executing other tasks. The I²C peripheral notifies the CPU through interrupts whenever software intervention is required.

By the end of this lecture, you will understand:

Why interrupt-driven communication is preferred.

The interrupt architecture of the STM32L452RE I²C peripheral.

The difference between polling and interrupts.

The role of the NVIC and the I²C interrupt service routine (ISR).

The overall interrupt-driven transfer flow.

Polling vs Interrupts

Polling Method

CPU

↓

Generate START

↓

Wait for TXIS

↓

Send Byte

↓

Wait for TXIS

↓

Send Byte

↓

Wait for TC

↓

Generate STOP

↓

Return

The CPU is busy throughout the transfer.

Interrupt Method

CPU

↓

Configure Transfer

↓

Generate START

↓

Continue Other Tasks

↓

Interrupt Occurs

↓

ISR Executes

↓

Return to Main Program

The CPU performs useful work while the I²C hardware manages the transfer.

Why Use Interrupts?

Polling wastes processing time.

Example:

TXIS Waiting

↓

CPU Doing Nothing

↓

TXIS Becomes Set

↓

Continue

Interrupts eliminate this idle waiting.

Benefits:

Better CPU utilization.

Supports multitasking and RTOS environments.

Lower power consumption.

Improved responsiveness.

```
Interrupt Architecture
          +----------------------+
          |   I²C Peripheral     |
```

```
    +-----------------------+
              |
  Status Flag Set (TXIS/RXNE/TC)
              |
              ▼
   Interrupt Request (IRQ)
              |
              ▼

           NVIC
              |
              ▼
  I2C1_EV_IRQHandler()
              |
              ▼
    Driver Interrupt Handler
              |
              ▼
     Continue Transfer
```

Types of I²C Interrupts

The STM32L452RE generates interrupts for several events.

Interrupt Source Meaning

TXIS TXDR ready for next byte

RXNE New byte received

TC Transfer complete

STOPF STOP condition detected

NACKF Slave did not acknowledge

ERRI Error interrupt (BERR, ARLO, OVR, TIMEOUT)

Registers Used

Register Purpose

CR1 Enable interrupt sources

CR2 Configure transfer

ISR Read interrupt flags

ICR Clear interrupt flags

Interrupt Enable Bits (CR1)

The CR1 register enables individual interrupt sources.

Bit Function

TXIE Transmit interrupt enable

RXIE Receive interrupt enable

TCIE Transfer complete interrupt

STOPIE STOP detection interrupt

NACKIE NACK interrupt

ERRIE Error interrupt

Example: Enable TX Interrupt

pI2CHandle->pI2Cx->CR1 |= I2C_CR1_TXIE;

This allows the peripheral to generate an interrupt whenever TXIS becomes set.

```
Example: Enable RX Interrupt
pI2CHandle->pI2Cx->CR1 |= I2C_CR1_RXIE;
Example: Enable Error Interrupts
pI2CHandle->pI2Cx->CR1 |= I2C_CR1_ERRIE;
NVIC (Nested Vectored Interrupt Controller)
```

The I²C peripheral can request an interrupt, but the NVIC decides whether the CPU should respond.

The driver must enable the corresponding IRQ in the NVIC.

Peripheral

↓

IRQ Generated

↓

NVIC

↓

CPU Executes ISR

Typical Initialization Sequence

Initialize I²C

↓

Configure TIMINGR

↓

Enable Peripheral

↓

Enable I²C Interrupts

↓

Enable NVIC IRQ

↓

Application Ready

Interrupt-Driven Transfer Flow

Application

↓

I2C_MasterSendDataIT()

↓

Configure CR2

↓

Generate START

↓

Return Immediately

↓

TXIS Interrupt

↓

Send Byte

↓

TXIS Interrupt

↓

Send Next Byte

↓

TC Interrupt

↓

Generate STOP

↓

STOPF Interrupt

↓

Transfer Complete Callback

Notice that the application does not wait for the transfer to finish.

Polling vs Interrupt Driver

Polling Driver Interrupt Driver

CPU waits for every flag CPU continues executing

Simple implementation More complex

Wastes CPU cycles Efficient CPU usage

Easier to debug Better for multitasking

Good for simple applications Preferred in real-time systems

New APIs to be Developed

Unlike the blocking APIs, the interrupt driver will introduce:

```
I2C_Status_t I2C_MasterSendDataIT(
    I2C_Handle_t *pI2CHandle,
    uint8_t *pTxBuffer,
    uint32_t Len,
    uint16_t SlaveAddr);
I2C_Status_t I2C_MasterReceiveDataIT(
    I2C_Handle_t *pI2CHandle,
    uint8_t *pRxBuffer,
    uint32_t Len,
    uint16_t SlaveAddr);
```

These functions start the transfer and return immediately. The actual byte transfer is handled by the interrupt service routine.

Driver Components

To support interrupt-driven communication, we will implement:

I2C_MasterSendDataIT()

↓

I2C_MasterReceiveDataIT()

↓

I2C_EV_IRQHandler()

↓

I2C_ER_IRQHandler()

↓

Application Callback

Key Takeaways

Interrupt-driven I²C allows the CPU to perform other tasks while the peripheral manages data transfers.

The STM32L452RE I²C peripheral generates interrupts for events such as TXIS, RXNE, TC, STOPF, NACKF, and communication errors.

Interrupt sources are enabled using the CR1 register, while the NVIC enables the corresponding processor interrupt.

Unlike blocking APIs, the interrupt-driven APIs return immediately after starting the transfer.

The actual communication is completed inside the interrupt service routine (ISR), making the driver more efficient and suitable for multitasking systems.

LECTURE 10: Preparing the Driver for Interrupt Mode (STM32L452RE)

Objective

In the previous lecture, we understood how interrupt-driven communication works.

In this lecture, we prepare the driver so that an interrupt service routine (ISR) can continue an I²C transfer after the application starts it.

Unlike the blocking driver, where all variables exist only inside the function, an interrupt-driven driver must preserve the transfer state because the transfer is completed over multiple interrupt events.

By the end of this lecture, you will:

Extend the I2C_Handle_t structure.

Store transfer information inside the handle.

Define driver state macros.

Create the I2C_MasterSendDataIT() API.

Configure the transfer and enable interrupts.

Return immediately to the application.

Why Modify the Handle Structure?

Consider the blocking driver:

I2C_MasterSendData()

↓

Configure CR2

↓

Wait TXIS

↓

Write Byte

↓

Wait TC

↓

STOP

↓

Return

All variables remain inside the function.

However, in interrupt mode:

Application

↓

I2C_MasterSendDataIT()

↓

Returns Immediately

↓

TXIS Interrupt

↓

ISR Sends Byte

↓

Returns

↓

TXIS Interrupt

↓

ISR Sends Next Byte

The ISR must know:

Which buffer is being transmitted.

How many bytes remain.

Which peripheral generated the interrupt.

What transfer is currently active.

These variables cannot be local variables.

Extending the Handle Structure

Modify the handle definition.

```
typedef struct
{
    I2C_RegDef_t *pI2Cx;

    I2C_Config_t I2C_Config;

    uint8_t *pTxBuffer;

    uint8_t *pRxBuffer;

    uint32_t TxLen;

    uint32_t RxLen;

    uint16_t DevAddr;

    uint8_t TxRxState;

}I2C_Handle_t;
Purpose of Each Member
```

Member Purpose

pTxBuffer Pointer to transmit buffer

pRxBuffer Pointer to receive buffer

TxLen Remaining transmit bytes

RxLen Remaining receive bytes

DevAddr Slave address

TxRxState Current driver state

Why Store These Variables?

Suppose

Buffer

11 22 33 44

After the first interrupt,

Sent

11

Remaining

22 33 44

The ISR must know

Current buffer location

Remaining bytes

Otherwise,

the transfer cannot continue.

Driver States

The driver should always know its current state.

Define

```
#define I2C_READY            0
#define I2C_BUSY_IN_TX       1
#define I2C_BUSY_IN_RX       2
```

Driver State Diagram

READY

```
↓
```

Start Transmission

```
↓
```

BUSY_IN_TX

```
↓
```

Transfer Complete

```
↓
```

READY

Similarly,

READY

↓

Start Reception

↓

BUSY_IN_RX

↓

Transfer Complete

↓

READY

Why Driver States?

Without a state variable,

two applications could attempt

Application A

↓

Start TX

↓

Application B

↓

Start TX Again

Result:

Transfer corruption.

The state variable prevents this.

```
API Prototype
I2C_Status_t I2C_MasterSendDataIT(
        I2C_Handle_t *pI2CHandle,
        uint8_t *pTxBuffer,
        uint32_t Len,
        uint16_t SlaveAddr);
```

Step 1 – Check Driver State

Before starting,

verify that the peripheral is free.

if(pI2CHandle->TxRxState != I2C_READY)

{

return I2C_BUSY;

}

Step 2 – Save Transfer Information

Instead of local variables,

store everything inside the handle.

```
pI2CHandle->pTxBuffer = pTxBuffer;
```

```
pI2CHandle->TxLen = Len;
```

```
pI2CHandle->DevAddr = SlaveAddr;
```

Now the ISR can access these values.

```
Step 3 — Update Driver State
pI2CHandle->TxRxState = I2C_BUSY_IN_TX;
```

The driver now knows

a transmission is active.

Step 4 – Configure CR2

The configuration is identical to the blocking driver.

Program

Slave Address

Write Direction

NBYTES

AUTOEND

START

CR2

```
↓
```

SADD

```
↓
```

RD_WRN = 0

```
↓
```

NBYTES

```
↓
```

START

Step 5 – Enable Interrupts

Unlike polling,

interrupt mode enables hardware interrupts.

Enable

```
pI2CHandle->pI2Cx->CR1 |=
```

I2C_CR1_TXIE |

I2C_CR1_TCIE |

I2C_CR1_STOPIE |

I2C_CR1_NACKIE |

I2C_CR1_ERRIE;

Why These Interrupts?

Interrupt Purpose

TXIE Send next byte

TCIE Transfer completed

STOPIE STOP generated

NACKIE Detect missing ACK

ERRIE Detect bus errors

Step 6 – Generate START

Exactly like the blocking driver,

```
pI2CHandle->pI2Cx->CR2 |= I2C_CR2_START;
```

What Happens Next?

After START,

the function returns immediately.

Application

↓

Start Transfer

↓

Return

↓

CPU Executes Other Code

When TXIS becomes set,

the peripheral interrupts the CPU.

Complete API Flow

Application

↓

Check Driver State

↓

Store Handle Information

↓

Configure CR2

↓

Enable Interrupts

↓

Generate START

↓

Return

↓

TXIS Interrupt Occurs

Registers Used

Register	Purpose
CR1	Enable interrupts
CR2	Configure transfer

Complete Driver Flow

Application

↓

I2C_MasterSendDataIT()

↓

Save Buffer

↓

Save Length

↓

Save Address

↓

Configure CR2

↓

Enable Interrupts

↓

Generate START

↓

Return

↓

Interrupt Handles Transfer

Key Takeaways

Interrupt-driven transfers require the driver to preserve transfer information after the API returns, so additional members are added to I2C_Handle_t.

The handle stores the transmit/receive buffer pointers, remaining byte counts, slave address, and current driver state.

The TxRxState variable prevents multiple transfers from using the same peripheral simultaneously.

I2C_MasterSendDataIT() no longer performs the complete transmission. Instead, it saves the transfer context, configures the peripheral, enables the required interrupts, generates the START condition, and returns immediately.

From this point onward, the interrupt service routine (ISR) is responsible for sending data, detecting transfer completion, handling STOP, and reporting errors.

LECTURE 11: Implementing the I²C Event Interrupt Handler (STM32L452RE)

Objective

In the previous lecture, we implemented I2C_MasterSendDataIT(), which configures the transfer and returns immediately after enabling interrupts.

However, no data is transmitted yet. The actual transfer is performed by the I²C Event Interrupt Handler, which responds to hardware-generated interrupt events.

By the end of this lecture, you will:

Understand how the I²C event interrupt works.

Implement the event interrupt handler.

Handle TXIS, TC, and STOPF events.

Complete an interrupt-driven transmit operation.

Interrupt Flow

Application

↓

I2C_MasterSendDataIT()

↓

Generate START

↓

Return

↓

TXIS Interrupt

↓

Event Handler

↓

Send Byte

↓

TXIS Interrupt

↓

Send Next Byte

↓

TC Interrupt

↓

Generate STOP

↓

STOPF Interrupt

↓

Transfer Finished

Which Interrupts Are Used?

Interrupt Purpose

TXIS TXDR ready for next byte

TC Transfer completed

STOPF STOP detected

NACKF Slave did not acknowledge

BERR Bus error

ARLO Arbitration lost

Driver Architecture

Hardware

↓

ISR Flags

↓

NVIC

↓

I2C1_EV_IRQHandler()

↓

I2C_EV_IRQHandler()

↓

Driver Actions

The MCU-specific interrupt function should be very small.

Example:

void I2C1_EV_IRQHandler(void)

{

I2C_EV_IRQHandler(&I2C1Handle);

}

This keeps the driver portable.

Generic Event Handler

Prototype

```
void I2C_EV_IRQHandler(I2C_Handle_t *pI2CHandle);
```

Inside this function, software checks the ISR register to determine which event occurred.

```
Reading the ISR Register
uint32_t isr = pI2CHandle->pI2Cx->ISR;
```

All event flags are available in this register.

Handling the TXIS Event

The first event after a successful address phase is usually TXIS.

Address ACK

```
↓
```

TXIS = 1

```
↓
```

Interrupt Generated

Detection:

if(isr & I2C_ISR_TXIS)

{

...

}

What Happens During TXIS?

If bytes remain to be transmitted,

TXIS

```
↓
```

```
Write TXDR
```

```
↓
```

Update Pointer

```
↓
```

Decrease Length

Implementation

```
if(pI2CHandle->TxLen > 0)
{
    pI2CHandle->pI2Cx->TXDR =
            *(pI2CHandle->pTxBuffer);

    pI2CHandle->pTxBuffer++;

    pI2CHandle->TxLen--;
}
```

The hardware automatically transmits the byte.

After transmission,

another TXIS interrupt is generated.

TXIS Flow

TXIS

↓

Write Byte

↓

TxLen--

↓

More Bytes?

↓

YES

↓

Next TXIS

↓

Repeat

Transfer Complete (TC)

When every byte has been transmitted,

the hardware sets

TC = 1

Interrupt generated.

Handling TC

Detection

if(isr & I2C_ISR_TC)

{

...

}

Action

```
pI2CHandle->pI2Cx->CR2 |= I2C_CR2_STOP;
```

Hardware generates the STOP condition.

STOP Detection

After STOP appears on the bus,

STOP Generated

↓

STOPF = 1

↓

Interrupt

Handling STOPF

if(isr & I2C_ISR_STOPF)

{

...

}

First,

clear the flag.

```
pI2CHandle->pI2Cx->ICR |= I2C_ICR_STOPCF;
Disable Interrupts
```

Since transmission is complete,

disable transmit-related interrupts.

```
pI2CHandle->pI2Cx->CR1 &= ~(
```

I2C_CR1_TXIE |

I2C_CR1_TCIE |

I2C_CR1_STOPIE |

I2C_CR1_NACKIE);

This prevents unnecessary interrupts after the transfer has finished.

```
Restore Driver State
pI2CHandle->TxRxState = I2C_READY;
```

Now another transfer may begin.

```
Clear Handle Variables
pI2CHandle->pTxBuffer = NULL;
```

```
pI2CHandle->TxLen = 0;
```

Resetting the handle avoids stale pointers.

Notify the Application

A callback function allows the application to know when the transfer is complete.

Prototype

```
__weak void I2C_ApplicationEventCallback(
            I2C_Handle_t *pI2CHandle,
            uint8_t Event)
{
```

}

Call it when the transfer finishes.

```
I2C_ApplicationEventCallback(
        pI2CHandle,
```

I2C_EVENT_TX_COMPLETE);

The application can override this weak function to perform custom actions.

Complete Event Handler Flow

Interrupt

↓

Read ISR

↓

TXIS?

↓

`Write TXDR`

↓

TC?

↓

Generate STOP

↓

STOPF?

↓

Clear STOP

↓

Disable Interrupts

↓

READY

↓

Callback

Registers Used

Register Purpose

ISR Read interrupt flags

TXDR Write transmit byte

CR2 Generate STOP

ICR Clear STOPF

CR1 Disable interrupts

Complete Event Sequence

START

↓

Address

↓

TXIS

↓

Byte 1

↓

TXIS

↓

Byte 2

↓

TXIS

↓

...

↓

TC

↓

STOP

↓

STOPF

↓

READY

↓

Application Callback

Important Code Fragments

```
TXIS Handler
if((isr & I2C_ISR_TXIS) && (pI2CHandle->TxLen > 0))
{
    pI2CHandle->pI2Cx->TXDR =
            *(pI2CHandle->pTxBuffer);

    pI2CHandle->pTxBuffer++;

    pI2CHandle->TxLen--;
}
```

```
TC Handler
if(isr & I2C_ISR_TC)
```

```
{
    pI2CHandle->pI2Cx->CR2 |= I2C_CR2_STOP;
}
```

```
STOPF Handler
if(isr & I2C_ISR_STOPF)
{
    pI2CHandle->pI2Cx->ICR |= I2C_ICR_STOPCF;

    pI2CHandle->TxRxState = I2C_READY;

    I2C_ApplicationEventCallback(
            pI2CHandle,

            I2C_EVENT_TX_COMPLETE);
}
```

Key Takeaways

The event interrupt handler performs the actual data transfer after I2C_MasterSendDataIT() returns.

The TXIS interrupt indicates that the TXDR register is ready for the next transmit byte.

When the TC flag is set, the ISR generates the STOP condition by setting the STOP bit in CR2.

After the STOP condition is transmitted, the STOPF interrupt is used to clear the STOP flag, disable transmit-related interrupts, restore the driver state to I2C_READY, and notify the application through a callback.

Separating the application API from the interrupt handler makes the driver non-blocking, allowing the CPU to execute other tasks while the I²C peripheral handles communication.

One important improvement

The implementation above works well for learning, but for a production-quality STM32L452RE driver, I recommend one refinement in the next lecture: split the logic into small helper functions such as:

I2C_MasterHandleTXISInterrupt()

I2C_MasterHandleTCInterrupt()

I2C_MasterHandleSTOPInterrupt()

I2C_MasterHandleNACKInterrupt()

I2C_MasterHandleErrorInterrupt()

This modular design closely matches professional embedded driver architectures and makes the code easier to maintain and extend (for example, when adding receive interrupts or DMA support).

LECTURE 12: Modular Event Interrupt Handling (STM32L452RE)

Objective

In the previous lecture, we implemented a single event interrupt handler that processed TXIS, TC, and STOPF events.

While functional, a large interrupt routine becomes difficult to maintain as more events such as RXNE, NACKF, and error conditions are added.

In this lecture, we refactor the interrupt driver into smaller helper functions. This modular approach improves readability, simplifies debugging, and makes future extensions—such as interrupt-driven receive and DMA—much easier.

By the end of this lecture, you will:

Refactor the event interrupt handler into dedicated helper functions.

Create separate handlers for TXIS, TC, STOPF, NACKF, and receive events.

Understand the benefits of modular interrupt design.

Why Modularize?

A single interrupt handler can quickly become difficult to read.

Interrupt

↓

TXIS?

↓

RXNE?

↓

TC?

↓

STOPF?

↓

NACK?

↓

Errors?

↓

Application Callback

As more events are added, the code becomes lengthy and harder to maintain.

Improved Driver Structure

Instead of one large function, use the following organization:

I2C1_EV_IRQHandler()

```
↓
```

I2C_EV_IRQHandler()

```
├── HandleTXIS()
├── HandleRXNE()
├── HandleTC()
├── HandleSTOP()
└── HandleNACK()
```

Each helper function has a single responsibility.

```
Helper Function Prototypes
static void I2C_MasterHandleTXISInterrupt(
        I2C_Handle_t *pI2CHandle);

static void I2C_MasterHandleTCInterrupt(
        I2C_Handle_t *pI2CHandle);

static void I2C_MasterHandleSTOPInterrupt(
        I2C_Handle_t *pI2CHandle);

static void I2C_MasterHandleRXNEInterrupt(
        I2C_Handle_t *pI2CHandle);

static void I2C_MasterHandleNACKInterrupt(
        I2C_Handle_t *pI2CHandle);
```

Using static limits these helper functions to the driver source file.

Updated Event Handler

The main ISR becomes much simpler.

```
void I2C_EV_IRQHandler(I2C_Handle_t *pI2CHandle)
{
    uint32_t isr = pI2CHandle->pI2Cx->ISR;

    if(isr & I2C_ISR_TXIS)
    {
        I2C_MasterHandleTXISInterrupt(pI2CHandle);
    }

    if(isr & I2C_ISR_RXNE)
    {
        I2C_MasterHandleRXNEInterrupt(pI2CHandle);
    }

    if(isr & I2C_ISR_TC)
    {
        I2C_MasterHandleTCInterrupt(pI2CHandle);
    }

    if(isr & I2C_ISR_STOPF)
    {
        I2C_MasterHandleSTOPInterrupt(pI2CHandle);
    }
```

```
    if(isr & I2C_ISR_NACKF)
    {
        I2C_MasterHandleNACKInterrupt(pI2CHandle);
    }
}
```

The ISR now acts as a dispatcher.

TXIS Handler

This function transmits one byte.

```
static void I2C_MasterHandleTXISInterrupt(
        I2C_Handle_t *pI2CHandle)
{
    if(pI2CHandle->TxLen > 0)
    {
        pI2CHandle->pI2Cx->TXDR =
                *(pI2CHandle->pTxBuffer);

        pI2CHandle->pTxBuffer++;

        pI2CHandle->TxLen--;
    }
}
```

TC Handler

When the programmed transfer completes, generate a STOP condition.

```
static void I2C_MasterHandleTCInterrupt(
        I2C_Handle_t *pI2CHandle)
{
    pI2CHandle->pI2Cx->CR2 |= I2C_CR2_STOP;
}
STOP Handler
```

This helper finalizes the transfer.

```
static void I2C_MasterHandleSTOPInterrupt(
        I2C_Handle_t *pI2CHandle)
{
    pI2CHandle->pI2Cx->ICR |= I2C_ICR_STOPCF;

    pI2CHandle->pI2Cx->CR1 &= ~(
```

I2C_CR1_TXIE |

I2C_CR1_TCIE |

I2C_CR1_STOPIE |

I2C_CR1_NACKIE);

```
    pI2CHandle->TxRxState = I2C_READY;

    pI2CHandle->pTxBuffer = NULL;

    pI2CHandle->TxLen = 0;

    I2C_ApplicationEventCallback(
            pI2CHandle,
```

```
            I2C_EVENT_TX_COMPLETE);
}
```

RXNE Handler

Although receive interrupts will be implemented in the next lecture, we can define the structure now.

```
static void I2C_MasterHandleRXNEInterrupt(
        I2C_Handle_t *pI2CHandle)
{
    if(pI2CHandle->RxLen > 0)
    {
        *(pI2CHandle->pRxBuffer) =
                pI2CHandle->pI2Cx->RXDR;

        pI2CHandle->pRxBuffer++;

        pI2CHandle->RxLen--;
    }
}
```

NACK Handler

A NACK indicates that the slave did not acknowledge either the address or a transmitted byte.

```
static void I2C_MasterHandleNACKInterrupt(
        I2C_Handle_t *pI2CHandle)
{
    pI2CHandle->pI2Cx->ICR |= I2C_ICR_NACKCF;

    pI2CHandle->TxRxState = I2C_READY;

    I2C_ApplicationEventCallback(
            pI2CHandle,

            I2C_EVENT_NACK);
}
```

Event Flow

Interrupt

↓

Read ISR

↓

Identify Event

↓

Call Corresponding Handler

↓

Return

Advantages of Modular Design

Single ISR Modular ISR

Long function Small helper functions

Difficult to debug Easier to debug

Hard to extend Easy to add new features

Poor readability Clear separation of responsibilities

Large switch/if blocks Dedicated handlers

Driver Architecture

Application

↓

I2C_MasterSendDataIT()

↓

Hardware Interrupt

↓

I2C1_EV_IRQHandler()

↓

I2C_EV_IRQHandler()

↓

Helper Functions

↓

Callback

↓

Application

Registers Used

Register Purpose

ISR Detect interrupt events

TXDR Write transmit data

RXDR Read received data

CR2 Generate STOP

CR1 Enable/disable interrupts

ICR Clear interrupt flags

Key Takeaways

Splitting the interrupt handler into helper functions improves code organization, readability, and maintainability.

The main event handler becomes a dispatcher that checks interrupt flags and invokes the appropriate helper function.

Each helper function performs a single task, such as transmitting a byte (TXIS), generating a STOP (TC), finalizing a transfer (STOPF), receiving a byte (RXNE), or handling a NACK.

This modular architecture closely matches the design used in professional embedded software and simplifies future additions such as interrupt-driven receive operations, DMA support, and advanced error handling.

LECTURE 13: Implementing I2C_MasterReceiveDataIT() (STM32L452RE)

Objective

In the previous lectures, we implemented interrupt-driven master transmission. In this lecture, we extend the driver to support interrupt-driven master reception.

```
Unlike transmission, where the CPU writes data to the TXDR register, reception requires the CPU to read da
```

By the end of this lecture, you will:

Implement the I2C_MasterReceiveDataIT() API.

Configure the peripheral for Read mode.

Enable receive-related interrupts.

Receive bytes through RXNE interrupts.

Complete the transfer using TC and STOPF.

Notify the application when reception finishes.

Interrupt-Driven Receive Flow

Application

↓

I2C_MasterReceiveDataIT()

↓

Configure CR2

↓

Generate START

↓

Return Immediately

↓

RXNE Interrupt

↓

`Read RXDR`

↓

Repeat

↓

TC Interrupt

↓

Generate STOP

↓

STOPF Interrupt

```
↓

Receive Complete Callback
API Prototype
I2C_Status_t I2C_MasterReceiveDataIT(
        I2C_Handle_t *pI2CHandle,
        uint8_t *pRxBuffer,
        uint32_t Len,
        uint16_t SlaveAddr);
```

Step 1 – Check Driver State

Ensure that no other transfer is active.

if(pI2CHandle->TxRxState != I2C_READY)

{

return I2C_BUSY;

}

Step 2 – Store Receive Information

The ISR will need access to the receive buffer and transfer details.

```
pI2CHandle->pRxBuffer = pRxBuffer;

pI2CHandle->RxLen = Len;

pI2CHandle->DevAddr = SlaveAddr;

pI2CHandle->TxRxState = I2C_BUSY_IN_RX;
```

Step 3 – Configure CR2

Configure the transfer:

Slave Address

Read Direction

Number of Bytes

Manual STOP (AUTOEND = 0)

CR2

```
↓
```

SADD

```
↓
```

RD_WRN = 1

```
↓
```

NBYTES

```
↓
```

AUTOENDE = 0

Example:

```
uint32_t tempreg = 0;

tempreg |= (SlaveAddr << 1);

tempreg |= I2C_CR2_RD_WRN;

tempreg |= (Len << I2C_CR2_NBYTES_Pos);

pI2CHandle->pI2Cx->CR2 = tempreg;
```

Step 4 – Enable Interrupts

For receive operations, enable the following interrupts.

```
pI2CHandle->pI2Cx->CR1 |=
```

I2C_CR1_RXIE |

I2C_CR1_TCIE |

I2C_CR1_STOPIE |

I2C_CR1_NACKIE |

I2C_CR1_ERRIE;

```
Step 5 – Generate START
pI2CHandle->pI2Cx->CR2 |= I2C_CR2_START;
```

The function now returns immediately.

RXNE Interrupt

Each time a byte is received:

Slave

```
↓

RXDR Updated

↓
```

RXNE = 1

```
↓
```

Interrupt

```
RXNE Handler
static void I2C_MasterHandleRXNEInterrupt(
        I2C_Handle_t *pI2CHandle)
{
    if(pI2CHandle->RxLen > 0)
    {
        *(pI2CHandle->pRxBuffer) =
                pI2CHandle->pI2Cx->RXDR;

        pI2CHandle->pRxBuffer++;
```

```
        pI2CHandle->RxLen--;
    }
}
```

```
Every interrupt reads one byte from RXDR.
```

Receive Flow

RXNE

↓

```
Read RXDR
```

↓

Store Byte

↓

RxLen--

↓

More Bytes?

↓

YES

↓

Next RXNE

Transfer Complete

When all programmed bytes have been received:

TC = 1

```
TC Handler
static void I2C_MasterHandleTCInterrupt(
        I2C_Handle_t *pI2CHandle)
{
    pI2CHandle->pI2Cx->CR2 |= I2C_CR2_STOP;
}
```

STOPF Handler

The STOP interrupt finalizes the receive operation.

```
static void I2C_MasterHandleSTOPInterrupt(
        I2C_Handle_t *pI2CHandle)
{
    pI2CHandle->pI2Cx->ICR |= I2C_ICR_STOPCF;

    pI2CHandle->pI2Cx->CR1 &= ~(
```

I2C_CR1_RXIE |

I2C_CR1_TCIE |

I2C_CR1_STOPIE |

I2C_CR1_NACKIE);

```
        pI2CHandle->TxRxState = I2C_READY;

        pI2CHandle->pRxBuffer = NULL;

        pI2CHandle->RxLen = 0;

        I2C_ApplicationEventCallback(
                pI2CHandle,

                I2C_EVENT_RX_COMPLETE);
    }
```

Complete Receive Sequence

START

↓

Address + Read

↓

RXNE

↓

Read Byte

↓

RXNE

↓

Read Byte

↓

...

↓

TC

↓

STOP

↓

STOPF

↓

READY

↓

Callback

Updated Event Handler

The dispatcher now supports both transmit and receive events.

```
void I2C_EV_IRQHandler(I2C_Handle_t *pI2CHandle)
{
    uint32_t isr = pI2CHandle->pI2Cx->ISR;

    if(isr & I2C_ISR_TXIS)
        I2C_MasterHandleTXISInterrupt(pI2CHandle);

    if(isr & I2C_ISR_RXNE)
        I2C_MasterHandleRXNEInterrupt(pI2CHandle);

    if(isr & I2C_ISR_TC)
        I2C_MasterHandleTCInterrupt(pI2CHandle);

    if(isr & I2C_ISR_STOPF)
        I2C_MasterHandleSTOPInterrupt(pI2CHandle);

    if(isr & I2C_ISR_NACKF)
        I2C_MasterHandleNACKInterrupt(pI2CHandle);
}
```

Registers Used

Register Purpose

CR1 Enable receive interrupts

CR2 Configure receive transfer

RXDR Read incoming data

ISR Monitor interrupt flags

ICR Clear STOP and NACK flags

Driver Flow

Application

↓

Receive API

↓

Configure CR2

↓

Enable RX Interrupt

↓

Generate START

↓

Return

↓

RXNE Interrupt

↓

`Read RXDR`

↓

TC

↓

STOP

↓

STOPF

↓

Application Callback

Key Takeaways

I2C_MasterReceiveDataIT() starts a non-blocking receive operation by saving the transfer context, configuring CR2 for Read mode, enabling receive-related interrupts, and generating the START condition.

Each RXNE interrupt indicates that a new byte is available in RXDR. The ISR reads the byte, stores it in the application buffer, advances the buffer pointer, and decrements the remaining byte count.

Once all requested bytes have been received, the TC interrupt generates the STOP condition.

The STOPF interrupt completes the transfer by clearing the STOP flag, disabling receive interrupts, restoring the driver state to I2C_READY, and notifying the application through a callback.

With both interrupt-driven transmit and receive implemented, the STM32L452RE driver now supports efficient non-blocking master communication.

Driver Status After This Lecture

You have now implemented:

□ I^{2}C Initialization

□ Blocking Master Transmit

□ Blocking Master Receive

□ Error Handling

□ Interrupt-driven Master Transmit

□ Interrupt-driven Master Receive

□ Modular Event Interrupt Handler

□ Application Callback Framework

The next logical topic is error interrupt handling (I2C_ER_IRQHandler), which will process BERR, ARLO, OVR, TIMEOUT, and NACK events in a dedicated error interrupt handler, making the driver suitable for robust real-world applications.

LECTURE 14: Implementing the I²C Error Interrupt Handler (I2C_ER_IRQHandler) (STM32L452RE)

Objective

In the previous lectures, the Event Interrupt Handler (I2C_EV_IRQHandler) managed normal communication events such as TXIS, RXNE, TC, and STOPF.

However, communication may fail because of:

Slave not acknowledging

Bus errors

Arbitration loss

Overrun

Timeout

The STM32L452RE provides a dedicated Error Interrupt to handle these conditions without mixing them with normal transfer events.

By the end of this lecture, you will:

Implement the Error Interrupt Handler.

Detect all major I²C errors.

Clear error flags correctly.

Restore the driver state.

Notify the application about the specific error.

Why Separate Event and Error Interrupts?

Instead of one large interrupt routine,

STM32 separates them.

```
                  I²C Peripheral
                        |
         ┌──────────────┴──────────────┐
         |                             |
  Event Interrupt               Error Interrupt

 (TXIS,RXNE,TC...)           (BERR,ARLO,OVR...)
         |                             |
         ▼                             ▼
 I2C_EV_IRQHandler()           I2C_ER_IRQHandler()
```

This keeps the driver modular and easier to maintain.

Error Flags

All error information is available in the ISR register.

Flag Description

NACKF No acknowledge received

BERR Bus error

ARLO Arbitration lost

OVR Overrun/Underrun

TIMEOUT Timeout detected

```
Error Handler Prototype
void I2C_ER_IRQHandler(I2C_Handle_t *pI2CHandle);
Read ISR Register
uint32_t isr = pI2CHandle->pI2Cx->ISR;
```

Now examine each error flag.

NACK Error

A NACK usually indicates

Wrong slave address

Device absent

Slave rejected data

Master

```
↓
```

Address

```
↓
```

No ACK

```
↓
```

NACKF = 1

Handle it

```
if(isr & I2C_ISR_NACKF)
{
    pI2CHandle->pI2Cx->ICR |= I2C_ICR_NACKCF;

    pI2CHandle->TxRxState = I2C_READY;

    I2C_ApplicationEventCallback(
            pI2CHandle,
```

I2C_EVENT_NACK);

}

Bus Error (BERR)

Occurs when an illegal START or STOP condition is detected on the bus.

```
if(isr & I2C_ISR_BERR)
{
    pI2CHandle->pI2Cx->ICR |= I2C_ICR_BERRCF;

    pI2CHandle->TxRxState = I2C_READY;

    I2C_ApplicationEventCallback(
            pI2CHandle,
```

I2C_EVENT_BERR);

}

Arbitration Lost (ARLO)

Occurs in multi-master systems.

Master A

```
↓
```

Master B

```
↓
```

Both Generate START

```
↓
```

One Loses Arbitration

```
↓
```

ARLO = 1

Implementation

```
if(isr & I2C_ISR_ARLO)
{
    pI2CHandle->pI2Cx->ICR |= I2C_ICR_ARLOCF;

    pI2CHandle->TxRxState = I2C_READY;

    I2C_ApplicationEventCallback(
            pI2CHandle,
```

I2C_EVENT_ARLO);

}

Overrun (OVR)

Occurs when

```
RXDR is not read in time.
TXDR is not updated before required.
if(isr & I2C_ISR_OVR)
{
    pI2CHandle->pI2Cx->ICR |= I2C_ICR_OVRCF;

    pI2CHandle->TxRxState = I2C_READY;

    I2C_ApplicationEventCallback(
            pI2CHandle,
```

I2C_EVENT_OVR);

}

Timeout Error

Occurs when

```
Clock stretching exceeds limits.
Communication stalls.
Bus remains busy too long.
if(isr & I2C_ISR_TIMEOUT)
{
    pI2CHandle->pI2Cx->ICR |= I2C_ICR_TIMOUTCF;

    pI2CHandle->TxRxState = I2C_READY;
```

```
    I2C_ApplicationEventCallback(
            pI2CHandle,

            I2C_EVENT_TIMEOUT);
}
```

Complete Error Handler

```
void I2C_ER_IRQHandler(I2C_Handle_t *pI2CHandle)
{
    uint32_t isr = pI2CHandle->pI2Cx->ISR;

    if(isr & I2C_ISR_NACKF)
    {
        pI2CHandle->pI2Cx->ICR |= I2C_ICR_NACKCF;
        pI2CHandle->TxRxState = I2C_READY;
        I2C_ApplicationEventCallback(
                pI2CHandle,

                I2C_EVENT_NACK);
    }

    if(isr & I2C_ISR_BERR)
    {
        pI2CHandle->pI2Cx->ICR |= I2C_ICR_BERRCF;
        pI2CHandle->TxRxState = I2C_READY;
        I2C_ApplicationEventCallback(
                pI2CHandle,

                I2C_EVENT_BERR);
    }

    if(isr & I2C_ISR_ARLO)
    {
        pI2CHandle->pI2Cx->ICR |= I2C_ICR_ARLOCF;
        pI2CHandle->TxRxState = I2C_READY;
        I2C_ApplicationEventCallback(
                pI2CHandle,

                I2C_EVENT_ARLO);
    }

    if(isr & I2C_ISR_OVR)
    {
        pI2CHandle->pI2Cx->ICR |= I2C_ICR_OVRCF;
        pI2CHandle->TxRxState = I2C_READY;
        I2C_ApplicationEventCallback(
                pI2CHandle,

                I2C_EVENT_OVR);
    }

    if(isr & I2C_ISR_TIMEOUT)
    {
        pI2CHandle->pI2Cx->ICR |= I2C_ICR_TIMOUTCF;
        pI2CHandle->TxRxState = I2C_READY;
        I2C_ApplicationEventCallback(
                pI2CHandle,

                I2C_EVENT_TIMEOUT);
    }
}
```

MCU-Specific IRQ Functions

The startup file should call the generic handlers.

void I2C1_EV_IRQHandler(void)

{

I2C_EV_IRQHandler(&I2C1Handle);

}

void I2C1_ER_IRQHandler(void)

{

I2C_ER_IRQHandler(&I2C1Handle);

}

Application Callback

The application receives notifications through a weak callback.

```
__weak void I2C_ApplicationEventCallback(
            I2C_Handle_t *pI2CHandle,
            uint8_t Event)
{
}
```

Example:

```
void I2C_ApplicationEventCallback(
            I2C_Handle_t *pI2CHandle,
            uint8_t Event)
{
    switch(Event)
    {
        case I2C_EVENT_TX_COMPLETE:
            break;

        case I2C_EVENT_RX_COMPLETE:
            break;

        case I2C_EVENT_NACK:
            break;

        case I2C_EVENT_BERR:
            break;

        case I2C_EVENT_ARLO:
            break;

        case I2C_EVENT_OVR:
            break;

        case I2C_EVENT_TIMEOUT:
            break;
    }
}
```

```
Complete Interrupt Architecture
                I²C Peripheral
                       │
         ┌─────────────┴─────────────┐
         │                           │
    Event IRQ                    Error IRQ
         │                           │
```

```
         ▼                              ▼
I2C_EV_IRQHandler()         I2C_ER_IRQHandler()
         |                              |
         └──────────────┬──────────────┘
                        ▼
        I2C_ApplicationEventCallback()
                        |
                        ▼
                 User Application
```

Driver Status

At this point your STM32L452RE driver supports:

I²C Driver

```
├── Initialization
├── GPIO Configuration
├── TIMINGR Configuration
├── Blocking Transmit
├── Blocking Receive
├── Error Handling (Polling)
├── Interrupt Transmit
├── Interrupt Receive
├── Event Interrupt Handler
├── Error Interrupt Handler
├── Application Callback
└── Driver State Management
```

Key Takeaways

The Event Interrupt Handler processes normal communication events, while the Error Interrupt Handler deals exclusively with communication faults.

Error conditions such as NACKF, BERR, ARLO, OVR, and TIMEOUT are detected through the ISR register and cleared using the corresponding bits in the ICR register.

Each error restores the driver state to I2C_READY and reports the event to the application through a callback.

Separating event and error handling results in a cleaner, more maintainable driver architecture that scales well as additional features are added.

Driver Completion

With this lecture, your STM32L452RE I²C driver is functionally complete for:

□ Polling mode

□ Interrupt mode

□ Event handling

□ Error handling

LECTURE 15: Introduction to DMA-Based I²C Communication (STM32L452RE)

Objective

Until now, we have developed two methods for I²C communication:

Polling (Blocking) Mode

Interrupt-Driven Mode

Although interrupts free the CPU from constantly polling status flags, the processor is still interrupted for every transmitted or received byte.

For large data transfers, this interrupt overhead becomes significant.

DMA (Direct Memory Access) solves this problem by transferring data directly between memory and the I²C peripheral without CPU intervention.

By the end of this lecture, you will understand:

What DMA is.

Why DMA is useful for I²C.

The DMA architecture of the STM32L452RE.

How DMA differs from polling and interrupts.

The complete DMA transfer sequence.

```
Three Methods of Data Transfer
                CPU Usage
```

Polling

Interrupt

DMA

Polling uses the CPU continuously.

Interrupts reduce CPU workload.

DMA minimizes CPU involvement.

What is DMA?

DMA stands for Direct Memory Access.

It is a hardware controller that transfers data directly between:

Memory and Peripheral

Peripheral and Memory

Memory and Memory

without requiring the CPU to move every byte.

CPU-Based Transfer

Without DMA:

Memory

↓

CPU

↓

TXDR

↓

I²C Peripheral

↓

Slave

Every byte passes through the CPU.

DMA Transfer

With DMA:

Memory

↓

DMA Controller

↓

TXDR

↓

I²C Peripheral

↓

Slave

The CPU only starts the transfer.

DMA handles the remaining data movement.

Receive Using DMA

Slave

↓

I²C Peripheral

↓

RXDR

↓

DMA

↓

Memory

Again,

the CPU is not involved in each byte transfer.

Why DMA?

Suppose

EEPROM Read

1024 Bytes

Polling

CPU

```
 ↓
 Read RXDR
 ↓
```

1024 Times

Interrupt

Interrupt

```
 ↓
 Read RXDR
 ↓
```

1024 Interrupts

DMA

Configure DMA

↓

Start Transfer

↓

Wait for Completion

Only one interrupt is generated at the end of the transfer.

STM32L452RE DMA Architecture

Memory

↓

DMA Channel

↓

I²C TXDR

↓

I²C Peripheral

Receive

I²C RXDR

↓

DMA Channel

↓

Memory

DMA Resources

The STM32L452RE includes:

DMA1

DMA2

Each DMA controller contains multiple channels that can be assigned to peripherals according to the device reference manual.

DMA Transfer Sequence

Application

↓

Configure DMA

↓

Configure I²C

↓

Enable DMA Request

↓

Generate START

↓

DMA Transfers Data

↓

Transfer Complete

↓

DMA Interrupt

↓

Application Callback

CPU Activity

Polling

CPU

↓

Every Byte

Interrupt

CPU

↓

Every Interrupt

DMA

CPU

↓

Transfer Start

↓

Transfer Complete

DMA Enable Bits

The STM32L452RE I²C peripheral enables DMA requests using the CR1 register.

Bit	Purpose
TXDMAEN	Enable transmit DMA requests
RXDMAEN	Enable receive DMA requests

Example

```
pI2CHandle->pI2Cx->CR1 |= I2C_CR1_TXDMAEN;
```

Receive

```
pI2CHandle->pI2Cx->CR1 |= I2C_CR1_RXDMAEN;
```

DMA Driver Components

Our DMA-capable I²C driver will eventually include:

I2C_MasterSendDataDMA()

↓

DMA Initialization

↓

Configure DMA Channel

↓

Enable TXDMAEN

↓

Generate START

↓

DMA Interrupt

↓

STOP

↓

Callback

Comparison

Feature Polling Interrupt DMA

CPU Load High Medium Very Low

Throughput Low Medium High

Complexity Simple Moderate High

Best For Small transfers General use Large transfers

When Should DMA Be Used?

DMA is most beneficial for:

EEPROM page programming

External Flash memory

OLED/LCD display updates

Audio data streaming

Sensor data logging

Large block transfers

For very small transfers (1–8 bytes), interrupts are often simpler and more efficient.

Key Takeaways

DMA transfers data directly between memory and the I²C peripheral without CPU involvement for each byte.

The CPU configures the transfer once and is notified only when the transfer completes or an error occurs.

The STM32L452RE enables DMA requests using the TXDMAEN and RXDMAEN bits in the CR1 register.

DMA provides the lowest CPU utilization and is ideal for large or continuous data transfers.

Course Status

At this stage, your STM32L452RE I²C driver covers:

 Peripheral initialization

 GPIO configuration

 TIMINGR configuration

 Blocking transmit/receive

 Polling error handling

 Interrupt transmit/receive

 Event interrupt handler

 Error interrupt handler

□ Callback framework

□ Introduction to DMA

From here, you can either continue with a

full DMA implementation, or move on to another peripheral such as

SPI,

UART, or

GPIO interrupt drivers.

Continue with a full DMA implementation

Absolutely. From here, we'll build a complete DMA-based I²C driver specifically for the STM32L452RE (I²C v2 + DMA controller). We'll ignore the STM32F4 implementation entirely and use the STM32L4 architecture.

The DMA portion is larger than the interrupt driver, so we'll build it in a structured sequence.

DMA Driver Roadmap (STM32L452RE)

Lecture 16 – DMA Fundamentals and STM32L452RE DMA Architecture

DMA controller overview

DMA1 and DMA2

Channels and request mapping

Memory-to-peripheral and peripheral-to-memory transfers

How I²C generates DMA requests

DMA transfer sequence

Lecture 17 – DMA Driver Initialization

We'll implement:

DMA_Init()

DMA_DeInit()

DMA_Enable()

DMA_Disable()

Topics:

RCC clock enable

Channel configuration

Transfer direction

Increment modes

Data size

Circular mode

Priority

Normal mode

Lecture 18 – Configuring DMA for I²C Transmission

Implement

I2C_MasterSendDataDMA()

Topics:

Configure DMA source address

Configure destination (TXDR)

Number of bytes

Enable TXDMAEN

Generate START

DMA automatically fills TXDR

Lecture 19 – DMA Transmission Complete

Topics:

DMA Transfer Complete interrupt

Disable DMA

Wait for TC

Generate STOP

Clear STOPF

Callback

Lecture 20 – DMA Reception

Implement

I2C_MasterReceiveDataDMA()

Topics

RXDMAEN

Peripheral source = RXDR

Memory destination

Automatic receive

Lecture 21 – DMA Receive Completion

Topics

DMA Complete interrupt

TC

STOP

Cleanup

Callback

Lecture 22 – DMA Error Handling

Topics

Transfer Error

FIFO Error (where applicable)

Timeout

I²C Errors

Recovery

Lecture 23 – Final DMA Driver

We'll integrate

I²C Driver

```
 ├── Blocking Driver
 ├── Interrupt Driver
 ├── DMA Driver
```

│ ├── TX DMA

│ ├── RX DMA

```
 │     ├── DMA IRQ
 │     ├── Error IRQ
 │     └── Callback
```

What You'll End Up With

After these lectures, your driver will support all three communication methods.

```
                    STM32L452RE I²C Driver

                    +----------------------+
                    |     Application      |
                    +----------------------+
                              |
         ┌────────────────────┼────────────────────┐
         │                    │                    │
         ▼                    ▼                    ▼
  Polling APIs         Interrupt APIs          DMA APIs
         │                    │                    │
         └────────────────────┼────────────────────┘
                              ▼
                      I²C Driver Core
                              │
               ┌──────────────┴──────────────┐
               ▼                             ▼
       Event/Error IRQ                    DMA IRQ
               │                             │
               └──────────────┬──────────────┘
                              ▼
```

STM32L452RE I²C

Course Status

By the end of the DMA section, you'll have implemented:

□ Blocking I²C Driver

□ Interrupt-Driven I²C Driver

□ DMA-Based I²C Driver (next)

□ Complete driver architecture

□ Application callbacks

□ Robust error handling

This will be a production-quality bare-metal I²C driver for the STM32L452RE, covering all major transfer modes.

LECTURE 16: DMA Fundamentals and STM32L452RE DMA Architecture

Objective

In the previous lectures, we developed polling and interrupt-driven I²C drivers. Although interrupt mode significantly reduces CPU workload, the processor is still interrupted for every transmitted or received byte.

DMA (Direct Memory Access) eliminates this overhead by transferring data directly between memory and the I²C peripheral. The CPU only configures the transfer and is notified when the entire transaction is complete.

By the end of this lecture, you will:

```
Understand the DMA controller in the STM32L452RE.
Learn how DMA interacts with the I²C peripheral.
Understand DMA request generation.
Learn the overall DMA transfer sequence.
Prepare for implementing the DMA driver.
Review of Previous Transfer Methods
Polling Mode
Application
      |
      ▼
I2C_MasterSendData()
      |
      ▼
Wait for TXIS
      |
      ▼
Write TXDR
      |
      ▼
Repeat for Every Byte
```

The CPU actively waits for each status flag.

```
Interrupt Mode
Application
      |
      ▼
I2C_MasterSendDataIT()
      |
      ▼
Return Immediately
      |
      ▼
TXIS Interrupt
      |
      ▼
Write TXDR
```

The CPU is interrupted once for every byte transferred.

```
DMA Mode
Application
```

```
        │
        ▼
 Configure DMA
        │
        ▼
 Generate START
        │
        ▼
 DMA Transfers Entire Buffer
        │
        ▼
 Transfer Complete Interrupt
```

Only one interrupt is typically generated after the complete transfer.

What is DMA?

DMA stands for Direct Memory Access.

Instead of the CPU copying every byte, the DMA controller performs the transfer autonomously.

The CPU only:

Configures DMA.

Starts the transfer.

Receives a completion interrupt.

CPU-Based Data Transfer

Without DMA

Memory

↓

CPU

↓

TXDR

↓

I²C Peripheral

↓

Slave Device

Each byte passes through the CPU.

DMA-Based Data Transfer

Memory

↓

DMA Controller

↓

TXDR

↓

I²C Peripheral

↓

Slave Device

The CPU is not involved in individual byte transfers.

DMA Reception

Slave

↓

I²C Peripheral

↓

RXDR

↓

DMA Controller

↓

Memory Buffer

DMA automatically stores received data into memory.

STM32L452RE DMA Controllers

The STM32L452RE includes two DMA controllers:

STM32L452RE

├── DMA1

└── DMA2

Each controller contains several independent channels that can be assigned to peripheral requests according to the STM32L452RE reference manual.

DMA Request Generation

The DMA controller does not continuously monitor peripherals.

Instead, peripherals generate DMA requests.

Example:

`TXDR Empty`

↓

I²C Generates DMA Request

↓

DMA Writes Next Byte

↓

`TXDR Full`

```
↓
```

Repeat

Similarly,

```
RXDR Full
```

```
↓
```

I²C Generates DMA Request

```
↓
```

```
DMA Reads RXDR
```

```
↓
```

Stores Byte in Memory

I²C DMA Enable Bits

DMA requests are enabled through the CR1 register.

Bit Description

TXDMAEN Enable transmit DMA requests

RXDMAEN Enable receive DMA requests

Transmit

```
pI2CHandle->pI2Cx->CR1 |= I2C_CR1_TXDMAEN;
```

Receive

```
pI2CHandle->pI2Cx->CR1 |= I2C_CR1_RXDMAEN;
```

When these bits are enabled, the I²C peripheral generates DMA requests automatically.

```
DMA Transfer Direction
Memory → Peripheral
```

Used during transmission.

Memory Buffer

```
↓
```

DMA

```
↓
```

TXDR

```
↓
```

```
I²C
Peripheral → Memory
```

Used during reception.

RXDR

```
↓
```

DMA

↓

Memory Buffer

DMA Transfer Sequence

Application

↓

Configure DMA

↓

Configure I²C

↓

Enable DMA Requests

↓

Generate START

↓

DMA Transfers All Bytes

↓

Transfer Complete

↓

DMA Interrupt

↓

Application Callback

CPU Utilization Comparison

Polling

CPU

Interrupt

CPU

DMA

CPU

DMA provides the lowest processor overhead.

Advantages of DMA

Minimal CPU usage.

Efficient large data transfers.

Higher system throughput.

Reduced interrupt frequency.

Ideal for continuous or high-volume data streams.

Typical Applications

DMA is especially useful for:

EEPROM page programming

External Flash memory

OLED and LCD display updates

Sensor data acquisition

Audio streaming

Data logging

Firmware updates

```
DMA Driver Architecture
                    Application
                         │
                         ▼
             I2C_MasterSendDataDMA()
                         │
             Configure DMA Channel
                         │
                         ▼
                 Configure I²C
                         │
                         ▼
              Enable TXDMAEN/RXDMAEN
                         │
                         ▼
                  Generate START
                         │
                         ▼
               DMA Moves Data
                         │
                         ▼
            DMA Transfer Complete IRQ
                         │
                         ▼

             Application Callback
```

Registers We'll Use

Register	Purpose
I2C_CR1	Enable DMA requests
I2C_CR2	Configure I²C transfer
I2C_TXDR	DMA transmit destination
I2C_RXDR	DMA receive source
DMA_CCRx	Configure DMA channel
DMA_CNDTRx	Number of data items
DMA_CPARx	Peripheral address
DMA_CMARx	Memory address
DMA_ISR	DMA interrupt status
DMA_IFCR	Clear DMA interrupt flags

Key Takeaways

DMA transfers data directly between memory and the I²C peripheral, eliminating per-byte CPU involvement.

The STM32L452RE uses dedicated DMA controllers with configurable channels to service peripheral requests.

The I²C peripheral generates DMA requests automatically when TXDMAEN or RXDMAEN is enabled.

DMA supports both Memory-to-Peripheral (transmit) and Peripheral-to-Memory (receive) transfers.

Compared with polling and interrupt-driven communication, DMA provides the highest efficiency for large or continuous data transfers.

Next Lecture (Lecture 17)

We'll begin writing the DMA driver itself by implementing:

DMA_Init()

DMA_DeInit()

DMA_Enable()

DMA_Disable()

You'll learn how to configure:

DMA clock

DMA channel

Transfer direction

Memory increment mode

Peripheral increment mode

Data width

Transfer priority

Normal mode

LECTURE 17: DMA Driver Initialization (STM32L452RE)

Objective

In the previous lecture, we studied the DMA architecture and how the I²C peripheral generates DMA requests.

In this lecture, we begin implementing a generic DMA driver that can later be used not only by the I²C driver but also by SPI, USART, ADC, and other peripherals.

By the end of this lecture, you will:

Create DMA configuration structures.

Implement DMA_Init().

Implement DMA_DeInit().

Implement DMA_Enable() and DMA_Disable().

Configure a DMA channel for future I²C transfers.

DMA Driver Files

Create two new files.

Drivers/

```
├── Inc/
│      DMA_Driver.h
│
└── Src/
       DMA_Driver.c
DMA Handle Structure
```

Create a handle similar to the I²C driver.

```
typedef struct
{
    DMA_Channel_TypeDef *pDMAChannel;

    DMA_Config_t DMA_Config;

}DMA_Handle_t;
DMA Configuration Structure
typedef struct
{
    uint32_t Direction;

    uint32_t Priority;

    uint32_t MemoryIncrement;

    uint32_t PeripheralIncrement;

    uint32_t MemoryDataAlignment;

    uint32_t PeripheralDataAlignment;

    uint32_t Mode;
```

}DMA_Config_t;

Configuration Parameters

Parameter Purpose

Direction TX or RX

Priority DMA priority

MemoryIncrement Increment memory pointer

PeripheralIncrement Increment peripheral address

MemoryDataAlignment Byte/Halfword/Word

PeripheralDataAlignment Byte/Halfword/Word

Mode Normal or Circular

DMA Driver APIs

void DMA_Init(DMA_Handle_t *pDMAHandle);

```
void DMA_DeInit(DMA_Handle_t *pDMAHandle);
```

```
void DMA_Enable(DMA_Handle_t *pDMAHandle);
```

```
void DMA_Disable(DMA_Handle_t *pDMAHandle);
DMA Initialization Flow
Enable DMA Clock
```

```
↓
```

Disable Channel

```
↓
```

Configure CCR

```
↓
```

Configure Priority

```
↓
```

Configure Direction

```
↓
```

Configure Increment Modes

```
↓
```

Configure Data Width

```
↓
```

Ready

Step 1 – Disable DMA Channel

Before modifying DMA registers, the channel must be disabled.

```
pDMAHandle->pDMAChannel->CCR &= ~DMA_CCR_EN;
```

```
Step 2 — Clear Previous Configuration
pDMAHandle->pDMAChannel->CCR = 0;
```

Step 3 – Configure Direction

Memory-to-Peripheral

```
pDMAHandle->pDMAChannel->CCR |=
```

DMA_CCR_DIR;

Peripheral-to-Memory

Leave DIR cleared.

```
Direction Summary
Direction	DIR Bit
Peripheral → Memory	0
Memory → Peripheral	1
```

```
Step 4 — Configure Memory Increment
if(pDMAHandle->DMA_Config.MemoryIncrement)
{
    pDMAHandle->pDMAChannel->CCR |= DMA_CCR_MINC;
}
```

Normally,

Memory Increment should be enabled.

Why Memory Increment?

Suppose

Buffer

10 20 30 40

Without increment

DMA

```
↓
```

10

```
↓
```

10

```
↓
```

10

```
↓
```

10

With increment

DMA

```
↓
```

10

```
↓
```

20

```
↓
```

30

```
↓
```

40

Peripheral Increment

For I²C,

```
TXDR and RXDR remain fixed.
```

Therefore

DMA_CCR_PINC = 0

Peripheral Increment should remain disabled.

Step 5 – Configure Data Width

I²C transfers bytes.

DMA_CCR_MSIZE_8BIT

DMA_CCR_PSIZE_8BIT

Both memory and peripheral widths should be configured for 8-bit transfers.

Step 6 – Configure Priority

Example

```
pDMAHandle->pDMAChannel->CCR |=
```

DMA_CCR_PL_HIGH;

Priority options:

Priority Description

Low Lowest

Medium Normal

High High

Very High Highest

Step 7 – Configure Mode

Normal Mode

CCR &= ~DMA_CCR_CIRC;

Circular Mode

CCR |= DMA_CCR_CIRC;

For I²C,

Normal Mode is typically used.

```
Enable DMA Channel
void DMA_Enable(DMA_Handle_t *pDMAHandle)
{
    pDMAHandle->pDMAChannel->CCR |= DMA_CCR_EN;
}
Disable DMA Channel
void DMA_Disable(DMA_Handle_t *pDMAHandle)
{
    pDMAHandle->pDMAChannel->CCR &= ~DMA_CCR_EN;
}
Deinitialize DMA
void DMA_DeInit(DMA_Handle_t *pDMAHandle)
{
    pDMAHandle->pDMAChannel->CCR = 0;

    pDMAHandle->pDMAChannel->CNDTR = 0;

    pDMAHandle->pDMAChannel->CPAR = 0;
```

pDMAHandle->pDMAChannel->CMAR = 0;

}

DMA Initialization Sequence

DMA_Init()

↓

Disable Channel

↓

Clear CCR

↓

Configure Direction

↓

Configure Priority

↓

Configure Increment

↓

Configure Data Width

↓

Configure Mode

↓

Ready

Registers Used

Register Purpose

CCR DMA channel configuration

CNDTR Number of data items

CPAR Peripheral address

CMAR Memory address

Complete Driver Architecture

DMA Driver

```
├── DMA_Init()
├── DMA_DeInit()
├── DMA_Enable()
└── DMA_Disable()
```

These functions form the reusable foundation for all future DMA transfers.

Key Takeaways

A generic DMA driver separates DMA configuration from the I²C driver, making it reusable across multiple peripherals.

Before configuring a DMA channel, it must always be disabled.

The CCR register controls transfer direction, increment modes, priority, transfer mode, and data width.

For I²C transfers:

Memory Increment is enabled.

Peripheral Increment is disabled because TXDR and RXDR are fixed addresses.

8-bit data width is used.

Normal mode is preferred for finite transfers.

The DMA_Enable() and DMA_Disable() APIs provide a simple interface for controlling DMA channels.

Important Correction for STM32L452RE

One point needs to be addressed before continuing:

```
The lecture above uses DMA channel registers (CCR, CNDTR, CPAR, CMAR), which correspond to the basic DMA c
```

```
So, before implementing I2C_MasterSendDataDMA(), we need one additional lecture on DMAMUX configuration. T
```

LECTURE 18: DMAMUX Configuration for STM32L452RE

Objective

In the previous lecture, we configured the DMA channel itself. However, a DMA channel does not automatically know which peripheral should generate its transfer requests.

```
The STM32L452RE introduces a DMA Request Multiplexer (DMAMUX) between peripherals and the DMA controller.
```

By the end of this lecture, you will:

```
Understand the role of the DMAMUX.
Learn how DMA requests are routed.
Configure a DMAMUX channel.
Connect the I²C peripheral to a DMA channel.
Prepare for DMA-based I²C transfers.
```

Why is DMAMUX Needed?

```
Without DMAMUX:
```

Peripheral

```
↓
```

DMA Channel

This direct connection is fixed.

```
With DMAMUX:
```

Peripheral

```
↓
```

DMAMUX

```
↓
```

DMA Channel

The connection becomes programmable.

STM32L452RE Data Path

I²C1

```
             TX Request
                  |
                  ▼
            DMAMUX Channel
                  |
                  ▼
           DMA1 Channel x
                  |
                  ▼
               Memory
DMAMUX Architecture
```

STM32L452RE

```
        +---------------------------+
        |        DMAMUX             |
        |                           |
```

I^{2}C1_TX ─────────────────────┐ |

SPI1_TX ─────────────────────┤ |

USART1_TX─────────────────────┤ |

ADC1─────────────────────────┤ |

```
 TIM2───────────────────┘            |
        +---------------------------+
                    |
                    ▼
            DMA1 Channel
```

Each DMA channel selects one peripheral request source.

```
DMAMUX Registers

Each DMA channel has a corresponding DMAMUX Channel Configuration Register (CxCR).
```

Example:

```
DMAMUX1_Channel0

↓
```

CxCR

```
↓
```

DMAREQ_ID

The DMAREQ_ID field selects the peripheral request.

```
DMAMUX Configuration Register
31                        8 7         0
```

+--------------------------+------------+

| Reserved | DMAREQ_ID |

+--------------------------+------------+

The important field for basic DMA operation is:

DMAREQ_ID – Selects the peripheral request source.

Selecting the Request Source

The reference manual provides a table mapping peripherals to request IDs.

Example:

Peripheral

```
↓
```

DMA Request Number

↓

Program DMAREQ_ID

For example:

I²C1_TX

↓

DMAREQ_ID = (device-specific value)

Similarly:

I²C1_RX

↓

DMAREQ_ID = (device-specific value)

Note: The exact DMAREQ_ID values must be taken from the STM32L452RE Reference Manual (RM0394) for your specific device revision. Do not hard-code values from another STM32 family.

```
Configuring the DMAMUX
```

Suppose Channel 2 is used.

```
DMAMUX1_Channel2->CCR = DMAREQ_ID;
```

or

```
DMAMUX1_Channel2->CCR &= ~DMAMUX_CxCR_DMAREQ_ID;

DMAMUX1_Channel2->CCR |= DMAREQ_ID;
```

This connects the selected peripheral request to DMA Channel 2.

Driver API

Create a reusable helper.

```
void DMAMUX_ConfigRequest(
        DMAMUX_Channel_TypeDef *pChannel,
        uint32_t RequestID);
```

Implementation:

```
void DMAMUX_ConfigRequest(
        DMAMUX_Channel_TypeDef *pChannel,
        uint32_t RequestID)
{
    pChannel->CCR &= ~DMAMUX_CxCR_DMAREQ_ID;

    pChannel->CCR |= RequestID;
}
Example: Configure I²C1 TX
DMAMUX_ConfigRequest(
        DMAMUX1_Channel2,

        I2C1_TX_DMA_REQUEST);
Example: Configure I²C1 RX
DMAMUX_ConfigRequest(
        DMAMUX1_Channel3,
```

I2C1_RX_DMA_REQUEST);

Complete Initialization Sequence

Enable DMA Clock

↓

Configure DMA Channel

↓

```
Configure DMAMUX
```

↓

Configure Memory Address

↓

Configure Peripheral Address

↓

Enable DMA

↓

Enable TXDMAEN/RXDMAEN

↓

```
Generate START
Relationship Between DMAMUX and DMA
Application
```

↓

DMAMUX

↓

DMA Channel

↓

I²C Peripheral

```
The DMAMUX does not transfer data.
```

It only routes requests.

The DMA controller performs the actual data transfer.

Driver Structure

DMA Driver

```
├── DMA_Init()
├── DMA_Enable()
├── DMA_Disable()
├── DMA_DeInit()
└── DMAMUX_ConfigRequest()
```

Registers Used

```
Register	Purpose
DMAMUX_CxCR	Select peripheral request
```

DMA_CCR□DMA configuration
DMA_CNDTR□Transfer count
DMA_CPAR□Peripheral address
DMA_CMAR□Memory address
Transfer Sequence
Application

↓

Configure DMA

↓

Configure DMAMUX

↓

Enable DMA

↓

Enable I²C DMA

↓

START

↓

DMA Request

↓

DMA Transfer

↓

Transfer Complete

Key Takeaways
The STM32L452RE uses a DMAMUX to connect peripheral DMA requests to DMA channels.
Every DMA channel must be assigned a DMA request source before it can transfer data.
The DMAREQ_ID field in the DMAMUX Channel Configuration Register determines which peripheral generates req
The DMAMUX only routes requests—it does not move data. The DMA controller performs the actual memory-to-pe
Configuring the DMAMUX is a mandatory step before enabling DMA-based I²C communication.
Next Lecture (Lecture 19)

Now that the DMA channel and DMAMUX are configured, we'll implement the first DMA API:

I2C_MasterSendDataDMA()

This lecture will cover:

Configuring the DMA source (CMAR)

Configuring the destination (CPAR = &I2C1->TXDR)

Setting the transfer count (CNDTR)

Enabling the DMA channel

Enabling TXDMAEN in I2C_CR1

Generating the START condition

At that point, the DMA controller will begin transmitting data to the I²C peripheral automatically.

LECTURE 19: Implementing I2C_MasterSendDataDMA() (STM32L452RE)

Objective

In the previous lectures, we configured:

```
The DMA controller
The DMA channel
The DMAMUX request routing
```

Now we will combine everything to perform an actual DMA-based I²C transmission.

```
Unlike the interrupt driver, the CPU will not write each byte into the TXDR register. Instead, the DMA con
```

By the end of this lecture, you will:

Implement I2C_MasterSendDataDMA().

Configure DMA transfer parameters.

Enable DMA requests in the I²C peripheral.

Generate the START condition.

Start a non-blocking DMA transfer.

```
DMA Transmission Flow
Application
      │
      ▼
I2C_MasterSendDataDMA()
      │
      ▼
Configure DMA
      │
      ▼
Configure I²C
      │
      ▼
Enable DMA Channel
      │
      ▼
Generate START
      │
      ▼
DMA Writes TXDR Automatically
      │
      ▼
Transfer Complete Interrupt
API Prototype
I2C_Status_t I2C_MasterSendDataDMA(
        I2C_Handle_t *pI2CHandle,
        uint8_t *pTxBuffer,
        uint16_t Len,
        uint16_t SlaveAddr);
```

Step 1 – Check Driver State

if(pI2CHandle->TxRxState != I2C_READY)

{

return I2C_BUSY;

}

Only one transfer should be active at a time.

```
Step 2 – Store Transfer Information
pI2CHandle->pTxBuffer = pTxBuffer;

pI2CHandle->TxLen = Len;

pI2CHandle->DevAddr = SlaveAddr;

pI2CHandle->TxRxState = I2C_BUSY_IN_TX;
```

The handle keeps track of the ongoing transfer.

Step 3 – Configure DMA Memory Address

The source is the application's transmit buffer.

```
DMA_Channel->CMAR =
        (uint32_t)pTxBuffer;
```

Step 4 – Configure Peripheral Address

The destination is always the I²C transmit register.

```
DMA_Channel->CPAR =
        (uint32_t)&(pI2CHandle->pI2Cx->TXDR);
```

Notice that the peripheral address never changes.

```
Step 5 – Configure Transfer Count
DMA_Channel->CNDTR = Len;
```

The DMA controller now knows how many bytes to transfer.

DMA Configuration Summary

CMAR

```
↓
```

Application Buffer

CPAR

```
↓
```

I²C TXDR

CNDTR

```
↓
```

Number of Bytes

Step 6 – Configure CR2

Prepare the I²C peripheral.

```
uint32_t temp = 0;

temp |= (SlaveAddr << 1);

temp |= (Len << I2C_CR2_NBYTES_Pos);

pI2CHandle->pI2Cx->CR2 = temp;
```

The transfer is configured exactly as in polling and interrupt modes.

```
Step 7 — Enable I²C DMA Requests
pI2CHandle->pI2Cx->CR1 |=
```

I2C_CR1_TXDMAEN;

```
Now the peripheral will generate DMA requests whenever TXDR becomes empty.

Step 8 — Enable DMA Transfer Complete Interrupt
DMA_Channel->CCR |= DMA_CCR_TCIE;
```

This interrupt occurs after the final byte is transferred.

Step 9 – Enable DMA Channel

DMA_Enable(&I2C1_TX_DMA_Handle);

The DMA controller is now ready.

```
Step 10 — Generate START
pI2CHandle->pI2Cx->CR2 |=
```

I2C_CR2_START;

This starts the I²C transaction.

What Happens Next?

START

```
↓
```

Address

```
↓

TXDR Empty

↓
```

DMA Request

```
↓
```

DMA Writes Byte

```
↓

TXDR Empty Again

↓
```

DMA Request

```
↓
```

Repeat Until CNDTR = 0

```
The CPU never writes to TXDR.
```

Hardware Operation

Memory Buffer

```
↓
```

DMA

```
↓
```

TXDR

```
↓
```

Shift Register

```
↓
```

SDA Line

```
Each time TXDR becomes empty, the I²C peripheral requests another byte from DMA.
```

```
Complete API
I2C_Status_t I2C_MasterSendDataDMA(
        I2C_Handle_t *pI2CHandle,
        uint8_t *pTxBuffer,
        uint16_t Len,
        uint16_t SlaveAddr)
{
    if(pI2CHandle->TxRxState != I2C_READY)
        return I2C_BUSY;

    pI2CHandle->pTxBuffer = pTxBuffer;
    pI2CHandle->TxLen = Len;
    pI2CHandle->DevAddr = SlaveAddr;
    pI2CHandle->TxRxState = I2C_BUSY_IN_TX;

    DMA_Channel->CMAR =
            (uint32_t)pTxBuffer;

    DMA_Channel->CPAR =
            (uint32_t)&(pI2CHandle->pI2Cx->TXDR);

    DMA_Channel->CNDTR = Len;

    pI2CHandle->pI2Cx->CR1 |=
```

I2C_CR1_TXDMAEN;

```
    DMA_Enable(&I2C1_TX_DMA_Handle);

    pI2CHandle->pI2Cx->CR2 |=
```

I2C_CR2_START;

```
    return I2C_OK;
}
```

Driver Flow

Application

↓

SendDataDMA()

↓

Configure DMA

↓

Enable TXDMAEN

↓

Enable DMA

↓

START

↓

DMA Transfers Data

↓

DMA Interrupt

↓

STOP

↓

Callback

Registers Used

Register Purpose

CMAR Memory source address

CPAR Peripheral destination address

CNDTR Number of bytes

CCR DMA configuration

CR1 Enable TX DMA requests

CR2 Generate START

CPU Workload

Polling

CPU

↓

Write Every Byte

Interrupt

CPU

↓

Interrupt Every Byte

DMA

CPU

```
↓
```

Configure Once

```
↓
```

Wait

This is why DMA is preferred for large transfers.

```
Key Takeaways
I2C_MasterSendDataDMA() prepares both the DMA controller and the I²C peripheral before starting the transf
The DMA channel is configured with:
CMAR pointing to the application transmit buffer.
CPAR pointing to the TXDR register.
CNDTR containing the number of bytes to transfer.
Enabling TXDMAEN allows the I²C peripheral to generate DMA requests whenever TXDR becomes empty.
Once the DMA channel is enabled and the START condition is generated, the DMA controller automatically tra
The CPU is only involved in setup and completion, making DMA highly efficient for large data transfers.
```

Important STM32L452RE Note

```
For a production-quality STM32L452RE driver, the API should not use a generic DMA_Channel variable as show
```

```
DMA_Handle_t *pTxDMAHandle;
DMA_Handle_t *pRxDMAHandle;
```

This allows the same driver to support different DMA channels and multiple I²C peripherals without modification.

Next Lecture (Lecture 20)

We'll implement the DMA Transfer Complete Interrupt Handler, which will:

Detect the end of the DMA transfer.

Disable the DMA channel.

Disable TXDMAEN.

Wait for the I²C Transfer Complete (TC) condition.

Generate the STOP condition.

Restore the driver state.

Notify the application through a callback.

This completes the transmit side of the DMA-based I²C driver.

LECTURE 20: DMA Transfer Complete Interrupt Handler (STM32L452RE)

Objective

In the previous lecture, we started a DMA-based I²C transmission using I2C_MasterSendDataDMA().

Once the DMA transfers all bytes from memory to the I²C transmit register, it generates a DMA Transfer Complete Interrupt.

This lecture explains how to:

Handle the DMA Transfer Complete interrupt.

Disable the DMA channel.

Disable I²C DMA requests.

Wait for the I²C transfer to finish.

Generate the STOP condition.

Restore the driver state.

Notify the application.

Complete Transmission Sequence

Application

↓

I2C_MasterSendDataDMA()

↓

DMA Enabled

↓

START

↓

DMA Transfers Bytes

↓

DMA Transfer Complete

↓

DMA IRQ

↓

Wait for I²C TC

↓

Generate STOP

↓

STOPF

↓

Callback

Why DMA Completion Isn't the End

```
The DMA controller only copies data into TXDR.
```

Memory

↓

DMA

↓

TXDR

The I²C peripheral still needs to:

Move TXDR to the shift register.

Clock every bit onto the SDA line.

Generate ACK cycles.

Finish the last byte.

Therefore:

DMA TC

≠

I²C Transmission Finished

DMA Transfer Complete

When

CNDTR = 0

the DMA sets the Transfer Complete flag.

DMA

↓

Transfer Complete

↓

Interrupt

DMA IRQ Handler

The MCU interrupt handler is simple.

void DMA1_Channel2_IRQHandler(void)

{

DMA_TX_IRQHandler(&I2C1Handle);

}

```
Generic DMA Handler
void DMA_TX_IRQHandler(
        I2C_Handle_t *pI2CHandle);
```

Step 1 – Check Transfer Complete Flag

if(DMA_GetTransferCompleteFlag())

{

...

}

The exact flag depends on the DMA channel being used.

Step 2 – Clear DMA Flag

DMA_ClearTransferCompleteFlag();

Always clear the interrupt flag before returning.

```
Step 3 — Disable DMA Channel
DMA_Disable(
        pI2CHandle->pTxDMAHandle);
```

No further DMA transfers should occur.

```
Step 4 — Disable TX DMA Requests
pI2CHandle->pI2Cx->CR1 &=
```

~I2C_CR1_TXDMAEN;

The I²C peripheral will stop requesting DMA transfers.

Current Status

DMA Finished

↓

No More Memory Transfers

↓

I²C Still Sending Last Byte

Step 5 – Wait for TC Flag

DMA completion does not mean the bus is idle.

Wait for

```
ISR.TC = 1
while(!(pI2CHandle->pI2Cx->ISR &
```

I2C_ISR_TC));

This indicates that all bytes have actually left the peripheral.

```
Step 6 — Generate STOP
pI2CHandle->pI2Cx->CR2 |=
```

I2C_CR2_STOP;

The STOP condition is transmitted on the bus.

```
Step 7 — Wait for STOPF
while(!(pI2CHandle->pI2Cx->ISR &
```

I2C_ISR_STOPF));

```
Step 8 — Clear STOP Flag
pI2CHandle->pI2Cx->ICR |=
```

I2C_ICR_STOPCF;

```
Step 9 — Restore Driver State
pI2CHandle->TxRxState =
```

I2C_READY;

```
pI2CHandle->TxLen = 0;

pI2CHandle->pTxBuffer = NULL;
```

The driver is now ready for another transfer.

```
Step 10 — Notify Application
I2C_ApplicationEventCallback(
        pI2CHandle,
```

I2C_EVENT_TX_COMPLETE);

```
Complete DMA TX Handler
void DMA_TX_IRQHandler(
        I2C_Handle_t *pI2CHandle)
{
    DMA_ClearTransferCompleteFlag();

    DMA_Disable(
            pI2CHandle->pTxDMAHandle);

    pI2CHandle->pI2Cx->CR1 &=
```

~I2C_CR1_TXDMAEN;

```
    while(!(pI2CHandle->pI2Cx->ISR &
```

I2C_ISR_TC));

```
    pI2CHandle->pI2Cx->CR2 |=
```

I2C_CR2_STOP;

```
    while(!(pI2CHandle->pI2Cx->ISR &
```

I2C_ISR_STOPF));

```
    pI2CHandle->pI2Cx->ICR |=
```

I2C_ICR_STOPCF;

```
    pI2CHandle->TxRxState =
```

I2C_READY;

```
    pI2CHandle->TxLen = 0;

    pI2CHandle->pTxBuffer = NULL;

    I2C_ApplicationEventCallback(
            pI2CHandle,

            I2C_EVENT_TX_COMPLETE);
}
```

Complete DMA Transmission Flow

Application

↓

Configure DMA

↓

START

↓

DMA

↓

Transfer Complete

↓

DMA IRQ

↓

Disable DMA

↓

Wait for I²C TC

↓

STOP

↓

STOPF

↓

READY

↓

Callback

Registers Used

Register Purpose

CCR Enable/Disable DMA channel

CNDTR Transfer counter

CR1 Enable/Disable TX DMA requests

CR2 Generate STOP

ISR Check TC and STOPF

ICR Clear STOP flag

DMA vs I²C Completion

DMA TC

```
↓
TXDR Empty
↓
```

Last Byte Still Transmitting

```
↓
```

I²C TC

```
↓
```

STOP

```
↓
```

STOPF

```
↓
```

Finished

This distinction is essential for correct operation.

Key Takeaways

A DMA Transfer Complete interrupt indicates that the DMA controller has finished copying all data from memory to TXDR, not that the I²C bus transaction has completed.

After disabling the DMA channel and clearing TXDMAEN, software must wait for the I²C Transfer Complete (TC) flag before generating the STOP condition.

The STOPF flag confirms that the STOP condition has been transmitted and should be cleared using the STOPCF bit in the ICR register.

Once the transfer is complete, the driver restores its internal state to I2C_READY and notifies the application using the callback mechanism.

Professional Driver Improvement

The implementation above uses polling inside the DMA interrupt handler for clarity. In a production-quality STM32L452RE driver, a better design is:

DMA Transfer Complete IRQ:

Disable DMA.

Disable TXDMAEN.

Return immediately.

I²C Event IRQ:

Detect the TC flag.

Generate STOP.

Handle STOPF.

Invoke the application callback.

This avoids busy-waiting inside an interrupt service routine and provides lower interrupt latency. It is the architecture commonly used in professional STM32 driver libraries.

Next Lecture (Lecture 21)

We'll implement DMA-based reception using:

I2C_MasterReceiveDataDMA()

This will include:

Configuring CMAR as the receive buffer.

Setting CPAR to RXDR.

Enabling RXDMAEN.

Receiving data automatically through DMA.

Completing the transfer with DMA and I²C interrupts.

After Lecture 21, both DMA Transmit and DMA Receive paths will be implemented.

LECTURE 21: Implementing I2C_MasterReceiveDataDMA() (STM32L452RE)

Objective

In the previous lecture, we implemented DMA-based master transmission. In this lecture, we implement DMA-based master reception.

```
During a receive operation, the I²C peripheral generates a DMA request whenever a new byte is available in
```

By the end of this lecture, you will:

Implement I2C_MasterReceiveDataDMA().

Configure DMA for Peripheral-to-Memory transfers.

Enable I²C receive DMA requests.

Start a DMA-based receive operation.

Understand the complete receive sequence.

DMA Receive Flow

Application

```
↓
```

I2C_MasterReceiveDataDMA()

```
↓
```

Configure DMA

```
↓
```

Configure I²C

```
↓
```

Enable RXDMAEN

```
↓
```

Generate START

```
↓
```

Slave Sends Data

```
↓
```

```
DMA Reads RXDR
```

```
↓
```

Stores Data in Memory

```
↓
```

```
DMA Complete Interrupt
API Prototype
I2C_Status_t I2C_MasterReceiveDataDMA(
        I2C_Handle_t *pI2CHandle,
        uint8_t *pRxBuffer,
```

```
        uint16_t Len,
        uint16_t SlaveAddr);
```

Step 1 – Verify Driver State

if(pI2CHandle->TxRxState != I2C_READY)

{

return I2C_BUSY;

}

Only one transfer can be active at a time.

```
Step 2 — Save Transfer Information
pI2CHandle->pRxBuffer = pRxBuffer;

pI2CHandle->RxLen = Len;

pI2CHandle->DevAddr = SlaveAddr;

pI2CHandle->TxRxState = I2C_BUSY_IN_RX;
```

Step 3 – Configure DMA Memory Address

Destination buffer

```
pI2CHandle->pRxDMAHandle
          ->pDMAChannel->CMAR =
          (uint32_t)pRxBuffer;
```

Step 4 – Configure Peripheral Address

Source register

```
pI2CHandle->pRxDMAHandle
          ->pDMAChannel->CPAR =
          (uint32_t)&(pI2CHandle->pI2Cx->RXDR);
```

```
Step 5 — Configure Transfer Count
pI2CHandle->pRxDMAHandle
          ->pDMAChannel->CNDTR = Len;
```

DMA Configuration Summary

RXDR

↓

CPAR

↓

DMA

↓

CMAR

↓

Receive Buffer

Step 6 – Configure CR2

Configure:

Slave Address

Read Direction

Number of Bytes

uint32_t temp = 0;

```
temp |= (SlaveAddr << 1);
```

```
temp |= I2C_CR2_RD_WRN;
```

```
temp |= (Len << I2C_CR2_NBYTES_Pos);
```

```
pI2CHandle->pI2Cx->CR2 = temp;
```

```
Step 7 — Enable RX DMA Requests
pI2CHandle->pI2Cx->CR1 |=
```

I2C_CR1_RXDMAEN;

```
The I²C peripheral now requests DMA whenever RXDR contains a new byte.
```

```
Step 8 — Enable DMA Interrupt
pI2CHandle->pRxDMAHandle
          ->pDMAChannel->CCR |=
```

DMA_CCR_TCIE;

```
Step 9 — Enable DMA Channel
DMA_Enable(
    pI2CHandle->pRxDMAHandle);
```

```
Step 10 — Generate START
pI2CHandle->pI2Cx->CR2 |=
```

I2C_CR2_START;

Reception now begins.

Hardware Operation

Slave

```
↓
```

RXDR

```
↓
```

DMA Request

```
↓
```

```
DMA Reads RXDR
```

```
↓
```

Memory Buffer

The CPU is not involved in individual byte transfers.

DMA Receive Sequence

START

```
↓
```

Address + Read

```
↓

RXDR Full

↓
```

DMA Request

```
↓

DMA Reads RXDR

↓
```

Repeat

```
↓
```

CNDTR = 0

```
↓
```

DMA Interrupt

```
Complete API
I2C_Status_t I2C_MasterReceiveDataDMA(
        I2C_Handle_t *pI2CHandle,
        uint8_t *pRxBuffer,
        uint16_t Len,
        uint16_t SlaveAddr)
{
    if(pI2CHandle->TxRxState != I2C_READY)
        return I2C_BUSY;

    pI2CHandle->pRxBuffer = pRxBuffer;
    pI2CHandle->RxLen = Len;
    pI2CHandle->DevAddr = SlaveAddr;
    pI2CHandle->TxRxState = I2C_BUSY_IN_RX;

    pI2CHandle->pRxDMAHandle->pDMAChannel->CMAR =
            (uint32_t)pRxBuffer;

    pI2CHandle->pRxDMAHandle->pDMAChannel->CPAR =
            (uint32_t)&(pI2CHandle->pI2Cx->RXDR);

    pI2CHandle->pRxDMAHandle->pDMAChannel->CNDTR =
            Len;

    pI2CHandle->pI2Cx->CR1 |=
```

I2C_CR1_RXDMAEN;

```
    DMA_Enable(
            pI2CHandle->pRxDMAHandle);

    pI2CHandle->pI2Cx->CR2 |=
```

I2C_CR2_START;

```
    return I2C_OK;
}
```

Driver Flow

Application

↓

ReceiveDataDMA()

↓

Configure DMA

↓

Enable RXDMAEN

↓

START

↓

```
DMA Reads RXDR
```

↓

Memory Buffer

↓

DMA Interrupt

↓

STOP

↓

Callback

Registers Used

Register Purpose

CMAR Receive buffer address

CPAR RXDR address

CNDTR Number of bytes

CCR DMA configuration

CR1 Enable RX DMA requests

CR2 Configure receive transfer

Memory Transfer Example

Suppose the slave sends:

11 22 33 44

DMA automatically performs:

RXDR

```
↓
11 → Buffer[0]
↓
22 → Buffer[1]
↓
33 → Buffer[2]
↓
44 → Buffer[3]
```

No software copy is required.

Key Takeaways

I2C_MasterReceiveDataDMA() configures a Peripheral-to-Memory DMA transfer.

The DMA channel is configured with:

CPAR pointing to the RXDR register.

CMAR pointing to the application receive buffer.

CNDTR containing the number of bytes to receive.

Setting RXDMAEN enables the I²C peripheral to generate DMA requests whenever a new byte is received.

The DMA controller automatically copies each byte from RXDR into memory, eliminating per-byte CPU intervention.

After the last byte is transferred, the DMA controller generates a Transfer Complete interrupt.

Professional STM32L452RE Design Note

For receive operations, the DMA controller may complete its transfer before the I²C peripheral has fully finished the bus transaction. As with DMA transmit, software must still handle the I²C Transfer Complete (TC) and STOPF events before considering the operation finished.

A robust implementation therefore combines:

```
DMA Interrupt → Indicates memory transfer completion.
I²C Event Interrupt → Indicates protocol completion on the I²C bus.
```

This separation ensures that the application is notified only after the entire receive transaction has completed successfully.

Next Lecture (Lecture 22)

We'll implement the DMA Receive Transfer Complete Interrupt Handler, including:

DMA Transfer Complete interrupt processing.

Disabling the DMA channel.

Disabling RXDMAEN.

Waiting for the I²C Transfer Complete (TC) condition.

Generating the STOP condition.

Clearing STOPF.

Restoring the driver state.

Invoking the application callback.

After that lecture, both DMA Transmit and DMA Receive implementations will be fully functional.

LECTURE 22: DMA Receive Transfer Complete Interrupt Handler (STM32L452RE)

Objective

```
When the DMA controller has transferred all requested bytes from the I²C receive register (RXDR) into the
```

However, this does not necessarily mean that the I²C peripheral has finished the receive transaction on the bus. The driver must still complete the I²C protocol correctly before notifying the application.

By the end of this lecture, you will:

Handle the DMA receive completion interrupt.

Disable the DMA channel.

Disable I²C receive DMA requests.

Wait for the I²C transfer to finish.

Generate the STOP condition.

Restore the driver state.

Notify the application.

Complete Receive Flow

Application

↓

ReceiveDataDMA()

↓

START

↓

Slave Sends Bytes

↓

RXDR

↓

DMA Stores Buffer

↓

DMA Transfer Complete

↓

DMA IRQ

↓

Wait for I²C TC

↓

STOP

↓

STOPF

↓

READY

↓

Callback

DMA Completion

When

CNDTR = 0

the DMA controller raises the Transfer Complete interrupt.

This only means:

RXDR

↓

DMA

↓

Memory

Completed

The I²C peripheral may still be completing the last byte on the bus.

DMA RX Interrupt

Example

void DMA1_Channel3_IRQHandler(void)

{

DMA_RX_IRQHandler(&I2C1Handle);

}

```
Generic Handler
void DMA_RX_IRQHandler(
        I2C_Handle_t *pI2CHandle);
```

Step 1 – Check Transfer Complete Flag

if(DMA_GetTransferCompleteFlag())

{

...

```
}
```

Step 2 – Clear DMA Flag

DMA_ClearTransferCompleteFlag();

Always clear the interrupt source first.

```
Step 3 — Disable DMA Channel
DMA_Disable(
        pI2CHandle->pRxDMAHandle);
```

```
Step 4 — Disable RX DMA Requests
pI2CHandle->pI2Cx->CR1 &=
```

~I2C_CR1_RXDMAEN;

No additional DMA requests will be generated.

Current Status

DMA Finished

```
↓
```

Receive Buffer Complete

```
↓
```

I²C Peripheral Still Busy

```
Step 5 — Wait for Transfer Complete
while(!(pI2CHandle->pI2Cx->ISR &
```

I2C_ISR_TC));

The TC flag indicates that the I²C peripheral has finished the programmed transfer.

```
Step 6 — Generate STOP
pI2CHandle->pI2Cx->CR2 |=
```

I2C_CR2_STOP;

```
Step 7 — Wait for STOPF
while(!(pI2CHandle->pI2Cx->ISR &
```

I2C_ISR_STOPF));

```
Step 8 — Clear STOP Flag
pI2CHandle->pI2Cx->ICR |=
```

I2C_ICR_STOPCF;

```
Step 9 — Restore Driver State
pI2CHandle->TxRxState =
```

I2C_READY;

```
pI2CHandle->RxLen = 0;
```

```
pI2CHandle->pRxBuffer = NULL;
```

```
Step 10 — Notify the Application
```

```
I2C_ApplicationEventCallback(
        pI2CHandle,
```

I2C_EVENT_RX_COMPLETE);

```
Complete Receive Handler
void DMA_RX_IRQHandler(
        I2C_Handle_t *pI2CHandle)
{
    DMA_ClearTransferCompleteFlag();

    DMA_Disable(
            pI2CHandle->pRxDMAHandle);

    pI2CHandle->pI2Cx->CR1 &=
```

~I2C_CR1_RXDMAEN;

```
    while(!(pI2CHandle->pI2Cx->ISR &
```

I2C_ISR_TC));

```
    pI2CHandle->pI2Cx->CR2 |=
```

I2C_CR2_STOP;

```
    while(!(pI2CHandle->pI2Cx->ISR &
```

I2C_ISR_STOPF));

```
    pI2CHandle->pI2Cx->ICR |=
```

I2C_ICR_STOPCF;

```
    pI2CHandle->TxRxState =
```

I2C_READY;

```
    pI2CHandle->RxLen = 0;

    pI2CHandle->pRxBuffer = NULL;

    I2C_ApplicationEventCallback(
            pI2CHandle,

            I2C_EVENT_RX_COMPLETE);
}
```

Complete Receive Sequence

START

```
↓
```

Address + Read

```
↓

RXDR Full

↓
```

DMA Copies Byte

↓

Repeat

↓

DMA Complete

↓

DMA IRQ

↓

Disable DMA

↓

TC

↓

STOP

↓

STOPF

↓

READY

↓

Callback

Registers Used

Register Purpose

CCR DMA Enable/Disable

CNDTR Transfer Counter

CR1 Enable/Disable RX DMA

CR2 Generate STOP

ISR Read TC and STOPF

ICR Clear STOPF

DMA Receive Timeline

Slave

↓

RXDR

↓

DMA

↓

Memory

↓

DMA TC

↓

I²C TC

↓

STOP

↓

STOPF

↓

Application Callback

Notice that DMA TC occurs before I²C TC.

Driver State Diagram

READY

↓

ReceiveDataDMA()

↓

BUSY_IN_RX

↓

DMA Running

↓

DMA Complete

↓

I²C Complete

↓

READY

Key Takeaways

The DMA Receive Transfer Complete interrupt indicates that all requested data has been copied into the application's receive buffer.

The driver must still wait for the I^2C Transfer Complete (TC) flag before generating the STOP condition.

After clearing the STOPF flag, the driver resets its internal state and becomes ready for another transfer.

The application callback is invoked only after both the DMA transfer and the I^2C protocol have completed successfully.

This separation between DMA completion and I^2C completion ensures reliable and standards-compliant communication.

Complete DMA Driver Status

Your STM32L452RE I^2C driver now supports:

Feature Status

Peripheral Initialization □

GPIO Configuration □

TIMINGR Configuration □

Polling Transmission □

Polling Reception □

Interrupt Transmission □

Interrupt Reception □

Event Interrupt Handler □

Error Interrupt Handler □

DMA Transmission □

DMA Reception □

DMA TX Complete Handler □

DMA RX Complete Handler □

Callback Framework □

Next Lecture (Lecture 23): Production-Quality DMA Driver Integration

In the final DMA lecture, we'll refine the implementation into a reusable, production-quality driver by covering:

Integrating DMA with the existing I^2C handle.

Supporting multiple I^2C peripherals and DMA channels.

Centralized DMA interrupt handling.

Unified callback mechanism.

Error handling for DMA and I^2C.

Final driver architecture and API organization suitable for real embedded projects.

LECTURE 23: Production-Quality DMA Driver Integration (STM32L452RE)

Objective

Our DMA implementation currently demonstrates the correct transfer sequence, but a production-quality embedded driver should be:

Modular

Reusable

Scalable

Independent of a specific DMA channel

Independent of a specific I²C peripheral

In this lecture, we redesign the driver architecture to support multiple I²C peripherals and DMA channels cleanly.

Current Driver Structure

Application

```
↓
```

I2C Driver

```
↓
```

DMA Driver

```
↓
```

DMA Hardware

Currently, some DMA resources are assumed globally.

We will remove those assumptions.

Updated I²C Handle

Extend the I²C handle to include DMA handles.

```
typedef struct
{
    I2C_TypeDef *pI2Cx;

    I2C_Config_t I2C_Config;

    DMA_Handle_t *pTxDMAHandle;

    DMA_Handle_t *pRxDMAHandle;

    uint8_t *pTxBuffer;

    uint8_t *pRxBuffer;

    uint16_t TxLen;

    uint16_t RxLen;

    uint16_t DevAddr;
```

```
    uint8_t TxRxState;

} I2C_Handle_t;
```

Now every I²C peripheral knows which DMA channels it uses.

DMA Handle

The DMA handle should contain:

```
typedef struct
{
    DMA_Channel_TypeDef *pDMAChannel;

    DMAMUX_Channel_TypeDef *pDMAMUXChannel;

    DMA_Config_t DMA_Config;

} DMA_Handle_t;
```

The driver now manages both:

```
DMA Channel
DMAMUX Channel
```

Driver Relationships

Application

```
↓
```

I2C Handle

```
├── I²C Registers
├── TX DMA Handle
└── RX DMA Handle

↓
```

DMA Handle

```
├── DMA Channel
└── DMAMUX Channel
```

This architecture supports multiple peripherals without changing the driver code.

Generic DMA APIs

```
Avoid hard-coded DMA channels.
```

Instead of:

DMA1_Channel2->CCR

use

```
pDMAHandle->pDMAChannel->CCR
```

This makes the driver reusable.

```
Generic DMAMUX Configuration
void DMAMUX_ConfigRequest(
        DMA_Handle_t *pDMAHandle,
        uint32_t RequestID)
{
    pDMAHandle->pDMAMUXChannel->CCR = RequestID;
}
```

No channel numbers are hard-coded.

```
Generic DMA Enable
void DMA_Enable(
        DMA_Handle_t *pDMAHandle)
{
    pDMAHandle->pDMAChannel->CCR |=

            DMA_CCR_EN;
}
```

```
Generic DMA Disable
void DMA_Disable(
        DMA_Handle_t *pDMAHandle)
{
    pDMAHandle->pDMAChannel->CCR &=

            ~DMA_CCR_EN;
}
```

Updated DMA TX API

Instead of

```
DMA_Channel->CMAR = ...
```

use

```
pI2CHandle->pTxDMAHandle
          ->pDMAChannel->CMAR =
          (uint32_t)pTxBuffer;
```

Every DMA operation is routed through the handle.

```
Updated DMA RX API
pI2CHandle->pRxDMAHandle
          ->pDMAChannel->CMAR =
          (uint32_t)pRxBuffer;
```

Again, the driver is independent of the physical DMA channel.

Interrupt Routing

Instead of writing application code inside interrupt handlers:

DMA IRQ

```
↓
```

DMA Driver

```
↓
```

I²C Driver

```
↓
```

Application Callback

The application only receives high-level events.

```
Unified Callback
void I2C_ApplicationEventCallback(
```

```
        I2C_Handle_t *pHandle,
        uint8_t Event);
```

Possible events:

TX Complete

RX Complete

DMA Error

Bus Error

Arbitration Lost

NACK

Timeout

The application does not need to know which interrupt occurred.

Error Handling

Both DMA and I²C can generate errors.

DMA:

Transfer Error

Configuration Error

I²C:

NACK

Bus Error

Arbitration Lost

Overrun

Timeout

All errors should be reported through the same callback interface.

```
 Production Driver Architecture
                    Application
                         │
                         ▼
          I2C_ApplicationEventCallback()
                         │
                         ▼
               I²C Driver APIs
                         │
      ┌──────────────────┴──────────────────┐
      ▼                                     ▼
  Polling Engine                    Interrupt Engine
                                            │
                                            ▼
                                       DMA Engine
                                            │
```

```
          ▼
   DMA Driver Layer
          │
          ▼
    DMAMUX Layer
          │
          ▼
  STM32L452RE Hardware
```

Driver API Summary

Initialization

I2C_Init()

I2C_DeInit()

DMA_Init()

`DMAMUX_ConfigRequest()`

Communication

Polling

I2C_MasterSendData()

I2C_MasterReceiveData()

Interrupt

I2C_MasterSendDataIT()

I2C_MasterReceiveDataIT()

DMA

I2C_MasterSendDataDMA()

I2C_MasterReceiveDataDMA()

Interrupt Processing

I2C_EV_IRQHandler()

I2C_ER_IRQHandler()

DMA_TX_IRQHandler()

DMA_RX_IRQHandler()

Complete Driver State Machine

```
              READY
                │
       ┌────────┼────────┐
       ▼        ▼        ▼
    Polling   Interrupt   DMA
       │        │         │
       ▼        ▼         ▼

  BUSY_IN_TX / BUSY_IN_RX
                │
                ▼
        Transfer Complete
```

```
│
▼
```

READY

Folder Organization

Drivers/

```
├── Inc/
│   ├── stm32l452xx.h
│   ├── I2C_Driver.h
│   ├── DMA_Driver.h
│   └── GPIO_Driver.h
│
└── Src/
    ├── I2C_Driver.c
    ├── DMA_Driver.c
    └── GPIO_Driver.c
```

This organization keeps each peripheral driver independent and maintainable.

Best Practices

Never hard-code DMA channels inside driver functions.

Keep DMA, DMAMUX, and I²C logic separated into dedicated layers.

Route all application notifications through callbacks.

Use handle structures to associate DMA resources with each I²C instance.

Avoid polling inside interrupt handlers in production code; use event-driven completion wherever possible.

Design APIs so they can be reused with other peripherals such as SPI, USART, and ADC.

Final Driver Capabilities

Your STM32L452RE I²C driver now supports:

Feature Status

GPIO Configuration □

Peripheral Initialization □

TIMINGR Configuration □

Polling Mode □

Interrupt Mode □

DMA Mode □

Master Transmission □

Master Reception □

Event Interrupt Handling □

Error Interrupt Handling □

DMA TX Completion □

DMA RX Completion □

Unified Callback Framework □

Modular DMA Integration □

DMAMUX Support

Course Completion

Congratulations! You have now built a complete bare-metal I^{2}C driver for the STM32L452RE, supporting:

Blocking (Polling) Transfers

Interrupt-Driven Transfers

DMA-Based Transfers

Event and Error Handling

DMAMUX Configuration

Reusable Driver Architecture

This provides a solid foundation for professional embedded software development and can be extended with additional features such as 10-bit addressing, SMBus, PEC, General Call, or multi-master support if your application requires them.